

Deploy Magento 2 bằng Jenkins

Guideline tổng hợp

Tháng 9, 2026

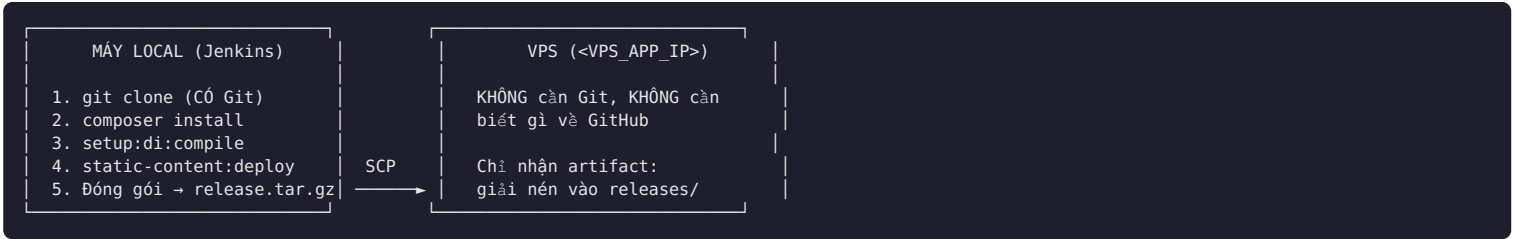
Deploy Magento 2 bằng Jenkins — Giai đoạn học: Jenkins chạy trên máy Local, SCM Polling

Bối cảnh: VPS chạy Magento 2 (Ubuntu 22.04, nginx/PHP-FPM/OpenSearch cài native), domain đã qua Cloudflare. Jenkins được cài tạm trên một máy local luôn bật, dùng để học và thử nghiệm pipeline trước khi chuyển sang chạy Jenkins ngay trên VPS với GitHub Webhook thật.

1. Quy ước placeholder dùng trong tài liệu

Placeholder	Ý nghĩa	Giá trị thực tế cần điền
<VPS_APP_IP>	IP VPS đang chạy Magento (nginx/php/opensearch)	IP VPS thật của bạn
<DOMAIN>	Domain đã trỏ qua Cloudflare	thenewxgear.com.vn
<DEPLOY_USER>	User Linux trên VPS dùng để deploy	deploy
<PROJECT_NAME>	Tên project	xgear
<PHP_FPM_SERVICE>	Tên service PHP-FPM trên VPS	ví dụ php8.1-fpm

2. Kiến trúc tổng thể



Nguyên tắc cốt lõi: build một lần (trên máy local), deploy artifact đã build sẵn — VPS không cần môi trường build (Composer/Node), không cần quyền truy cập GitHub, giảm tải và giảm bề mặt tấn công.

Chiến lược “releases + symlink”

```
/var/www/xgear-deploy/  
├── releases/  
│   ├── 20260925103000/ ← code mới nhất  
│   ├── 20260924180000/ ← release cũ (giữ lại để rollback)  
│   └── 20260923091500/  
├── shared/  
│   ├── app/etc/env.php ← config DB thật, không đổi giữa các lần deploy  
│   ├── pub/media/ ← ảnh sản phẩm khách hàng upload  
│   └── var/log/ ← log gộp xuyên suốt các lần deploy  
└── current -> releases/20260925103000 ← nginx trỏ vào symlink này
```

Nginx `root` / `$MAGE_ROOT` trỏ vào `current`. Mỗi lần deploy, chỉ cần đổi hướng symlink `current` bằng 1 lệnh `ln -sf` — gần như tức thời.

Lưu ý về thuật ngữ “zero-downtime”: pipeline này chính xác là **minimal-downtime deployment**, không phải zero-downtime tuyệt đối. Vẫn cần một khoảng bảo trì ngắn (vài giây đến vài chục giây) trong lúc chạy `setup:upgrade` (migration database), vì code mới và database phải đồng bộ schema cùng lúc. Zero-downtime thật sự (blue-green, rolling deploy) cần tối thiểu 2 server chạy song song với load balancer — phức tạp hơn nhiều, không cần thiết ở quy mô 1 VPS.

3. Chuẩn bị trên VPS (đích deploy)

3.1. Backup trước khi làm bất cứ gì

```
ssh root@-VPS_APP_IP>

sudo tar -czf /root/backup-xgear-$(date +%Y%m%d).tar.gz -C /var/www/html xgear
mysqldump -u root -p xgear_db > /root/backup-xgear-db-$(date +%Y%m%d).sql
```

3.2. Tạo user `deploy` riêng (không dùng root)

```
sudo adduser deploy
sudo usermod -aG www-data deploy
```

3.3. Tạo cấu trúc thư mục releases/shared

```
sudo mkdir -p /var/www/xgear-deploy/{releases,shared/app/etc,shared/pub/media,shared/var/log,shared/var/session,shared/var/composer_home}
sudo chown -R deploy:www-data /var/www/xgear-deploy
```

3.4. Copy dữ liệu “sống” từ project cũ sang `shared/`

```
sudo cp /var/www/html/xgear/app/etc/env.php /var/www/xgear-deploy/shared/app/etc/env.php
sudo rsync -a /var/www/html/xgear/pub/media/ /var/www/xgear-deploy/shared/pub/media/
sudo rsync -a /var/www/html/xgear/var/log/ /var/www/xgear-deploy/shared/var/log/
sudo chown -R deploy:www-data /var/www/xgear-deploy/shared
```

3.5. Tạo release đầu tiên từ code hiện tại (để `current` chạy ngay, không downtime lúc chuyển đổi)

```
RELEASE_NAME=$(date +%Y%m%d%H%M%S)
sudo mkdir -p /var/www/xgear-deploy/releases/$RELEASE_NAME
sudo rsync -a --exclude='pub/media' --exclude='var/log' --exclude='var/session' \
/var/www/html/xgear/ /var/www/xgear-deploy/releases/$RELEASE_NAME/

sudo rm -f /var/www/xgear-deploy/releases/$RELEASE_NAME/app/etc/env.php
sudo ln -sfn /var/www/xgear-deploy/shared/app/etc/env.php \
/var/www/xgear-deploy/releases/$RELEASE_NAME/app/etc/env.php

sudo rm -rf /var/www/xgear-deploy/releases/$RELEASE_NAME/pub/media
sudo ln -sfn /var/www/xgear-deploy/shared/pub/media \
/var/www/xgear-deploy/releases/$RELEASE_NAME/pub/media

sudo rm -rf /var/www/xgear-deploy/releases/$RELEASE_NAME/var/log
sudo ln -sfn /var/www/xgear-deploy/shared/var/log \
/var/www/xgear-deploy/releases/$RELEASE_NAME/var/log

sudo ln -sfn /var/www/xgear-deploy/releases/$RELEASE_NAME /var/www/xgear-deploy/current
sudo chown -R deploy:www-data /var/www/xgear-deploy
```

3.6. Sửa nginx trở sang `current`

Nếu dùng file mẫu chính thức Magento (`nginx.conf.sample` với biến `$MAGE_ROOT`):

```
server {
    listen 443 ssl;
    server_name thenewxgear.com.vn www.thenewxgear.com.vn;

    ssl_certificate /etc/letsencrypt/live/thenewxgear.com.vn/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/thenewxgear.com.vn/privkey.pem;

    set $MAGE_ROOT /var/www/xgear-deploy/current;
    include /var/www/xgear-deploy/current/nginx.conf.sample;

    error_log /var/log/nginx/xgear_error.log;
    access_log /var/log/nginx/xgear_access.log;
}
```

Chỉ cần đổi 2 dòng: `set $MAGE_ROOT` và `include`. File `nginx.conf.sample` bên trong mỗi release tự dùng biến `$MAGE_ROOT` cho `root`, `fastcgi_param`, `try_files`, nên không cần symlink riêng cho file này.

Kiểm tra và áp dụng:

```
sudo grep -rn "xgear" /etc/nginx/sites-available/ /etc/nginx/sites-enabled/ /etc/nginx/conf.d/ 2>/dev/null
sudo nginx -t
sudo systemctl reload nginx
```

Mở <https://<DOMAIN>> kiểm tra site vẫn chạy bình thường. Nếu lỗi, rollback ngay bằng cách đổi 2 dòng về đường dẫn cũ (`/var/www/html/xgear`) rồi `nginx -t && systemctl reload nginx`. **Giữ nguyên thư mục code cũ ít nhất 1-2 tuần** để có đường lùi.

3.7. Cấp sudo giới hạn cho user `deploy`

```
sudo visudo -f /etc/sudoers.d/deploy
```

```
deploy ALL=(ALL) NOPASSWD: /usr/sbin/service php8.1-fpm reload
deploy ALL=(ALL) NOPASSWD: /usr/bin/systemctl reload nginx
deploy ALL=(ALL) NOPASSWD: /usr/bin/chown -R www-data:www-data /var/www/xgear-deploy/releases/*
```

(Sửa `php8.1-fpm` đúng version đang dùng — kiểm tra bằng `php -v`.)

3.8. Bảo mật SSH (chỉ login bằng key, không dùng password)

```
sudo nano /etc/ssh/sshd_config
```

Đảm bảo có:

```
PasswordAuthentication no
PubkeyAuthentication yes
```

```
sudo systemctl restart sshd
```

4. Cài Jenkins trên máy Local

4.1. Cài Jenkins (native, Ubuntu)

```
sudo apt update
sudo apt install -y openjdk-17-jre

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins
```

Vì đây là máy local, Jenkins UI chỉ cần mở tại `http://localhost:8080` — không cần public ra Internet.

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Mở `http://localhost:8080` → cài Suggested plugins → tạo admin user.

4.2. Cài PHP + Composer trên máy local

Version PHP trên máy local nên **giống hệt** version PHP trên VPS để tránh lỗi tương thích khi compile.

```
ssh deploy@<VPS_APP_IP> "php -v" # kiểm tra version trên VPS trước
```

```
sudo apt install -y software-properties-common
sudo add-apt-repository ppa:ondrej/php
sudo apt update
sudo apt install -y php8.1 php8.1-cli php8.1-common php8.1-curl php8.1-mbstring \
  php8.1-xml php8.1-zip php8.1-bcmath php8.1-intl php8.1-soap php8.1-gd php8.1-mysql

php -v
```

```
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer
composer -V
```

4.3. Cài plugin Jenkins cần thiết

Manage Jenkins → Plugins → Available plugins, tick chọn: - Git - **SSH Agent** (bắt buộc, dùng `sshagent` trong Jenkinsfile) - Pipeline - Credentials Binding

Restart Jenkins nếu được yêu cầu.

5. Tạo SSH Key cho Jenkins → VPS

```
# Trên máy local, dưới quyền user thường
ssh-keygen -t ed25519 -C "jenkins-deploy-xgear" -f ~/jenkins_deploy_key -N ""

ssh-copy-id -i ~/jenkins_deploy_key.pub deploy@VPS_APP_IP>

# Test kết nối
ssh -i ~/jenkins_deploy_key deploy@VPS_APP_IP> "whoami && pwd"
```

Kết quả phải in ra `deploy` — nếu bị hỏi password, kiểm tra lại việc copy key.

Đưa private key vào Jenkins Credentials

1. `http://localhost:8080` → **Manage Jenkins** → **Credentials** → **System** → **Global credentials** → **Add Credentials**
2. Điền:
 - **Kind:** SSH Username with private key
 - **ID:** `jenkins-deploy-key`
 - **Username:** `deploy`
 - **Private Key** → **Enter directly** → dán nội dung file (`cat ~/jenkins_deploy_key`), gồm cả 2 dòng `BEGIN / END`
3. **Create**

Jenkins tự lưu key vào kho Credentials riêng — không cần đặt key vào `~/.ssh` của user `jenkins`.

6. Kết nối Jenkins với Git repo + SCM Polling

6.1. Tạo credential Git (nếu repo private)

GitHub: Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token → scope `repo` → copy token.

Đưa vào Jenkins: Manage Jenkins → Credentials → Add Credentials: - Kind: Username with password - Username: username GitHub - Password: personal access token - ID: `git-xgear-token`

6.2. Tạo Pipeline Job

1. **New Item** → tên `deploy-xgear-local` → chọn **Pipeline** → OK
2. **Pipeline** → Definition: `Pipeline script from SCM`
 - SCM: `Git`
 - Repository URL: `https://github.com/<user>/<repo>.git`
 - Credentials: `git-xgear-token`
 - Branches to build: `*/main`
 - Script Path: `Jenkinsfile`

6.3. Bật SCM Polling

Build Triggers → tick **Poll SCM** → Schedule:

```
H/5 * * * *
```

Jenkins tự kiểm tra repo mỗi ~5 phút, tự build nếu có commit mới. Không cần mở port, không cần webhook — phù hợp cho giai đoạn học, an toàn tuyệt đối (không public gì ra Internet).

7. Chuẩn bị `env.php.jenkins` — cấu hình giả để compile

Vì sao cần

`setup:di:compile` và `static-content:deploy` cần file `app/etc/env.php` tồn tại, nhưng **không được** commit config DB production thật lên Git. Giải pháp: tạo bản mẫu với giá trị giả.

Vị trí: trong thư mục clone Git trên máy local (KHÔNG phải Jenkins workspace)

```
cd /var/www/xgear-source # thư mục bạn tự clone để soạn thảo
ls app/etc/
nano app/etc/env.php.jenkins
```

Nội dung mẫu:

```
<?php
return [
    'backend' => [
        'frontName' => 'admin'
    ],
    'cache' => [
        'frontend' => [
            'default' => [
                'backend' => 'Cm_Cache_Backend_File',
            ],
            'page_cache' => [
                'backend' => 'Cm_Cache_Backend_File',
            ]
        ]
    ],
    'db' => [
        'table_prefix' => '',
        'connection' => [
            'default' => [
                'host' => '127.0.0.1',
                'dbname' => 'magento_dummy',
                'username' => 'dummy_user',
                'password' => 'dummy_pass',
                'active' => '1'
            ]
        ]
    ],
    'crypt' => [
        'key' => 'dummy32characterkeyforcompileonly'
    ],
    'resource' => [
        'default_setup' => [
            'connection' => 'default'
        ]
    ],
    'session' => [
        'save' => 'files'
    ],
    'MAGE_MODE' => 'production',
    'cache_types' => [
        'config' => 1,
        'layout' => 1,
        'block_html' => 1,
        'collections' => 1,
        'reflection' => 1,
        'db_ddl' => 1,
        'compiled_config' => 1,
        'eav' => 1,
        'customer_notification' => 1,
        'config_integration' => 1,
        'config_integration_api' => 1,
        'full_page' => 1,
        'config_webservice' => 1,
        'translate' => 1
    ]
]
```

Nên đối chiếu với file `env.php` thật trên VPS để không thiếu key (chỉ xem, không copy giá trị thật):

```
ssh deploy@<VPS_APP_IP> "cat /var/www/xgear-deploy/shared/app/etc/env.php"
```

Nếu stage compile báo lỗi kết nối DB, cần nhắc cài MariaDB local thật với DB dummy:

```
sudo apt install -y mariadb-server
sudo mysql -e "CREATE DATABASE magento_dummy;"
sudo mysql -e "CREATE USER 'dummy_user'@'localhost' IDENTIFIED BY 'dummy_pass';"
sudo mysql -e "GRANT ALL ON magento_dummy.* TO 'dummy_user'@'localhost';"
```

Cập nhật `.gitignore` — chỉ chặn file thật, không chặn file mẫu

```
cat .gitignore | grep -n "env.php"
```

Đảm bảo có đúng dòng:

```
/app/etc/env.php
```

Không dùng wildcard `env.php*` (sẽ chặn nhầm cả `env.php.jenkins`).

Kiểm tra kỹ trước khi commit

```
git status
git check-ignore -v app/etc/env.php
```

`git status` phải thấy `env.php.jenkins` (untracked, cần add) nhưng **không** thấy `env.php` (không đuôi `.jenkins`). Lệnh `git check-ignore`

phải in ra kết quả xác nhận `env.php` đang bị ignore.

```
git add app/etc/env.php.jenkins .gitignore
git commit -m "Add env.php.jenkins template for Jenkins CI compile"
git push origin main
```

Kiểm tra chéo trên GitHub: vào `app/etc/` , chỉ thấy `env.php.jenkins` , tuyệt đối không thấy `env.php` thật.

8. Jenkinsfile hoàn chỉnh

Tạo tại thư mục gốc repo (`/var/www/xgear-source/Jenkinsfile` , cùng cấp `composer.json`):

```
pipeline {
    agent any

    environment {
        DEPLOY_HOST      = 'deploy@<VPS_APP_IP>'
        REMOTE_BASE      = '/var/www/xgear-deploy'
        RELEASE_NAME     = "${new Date().format('yyyyMMddHHmmss')}"
        RELEASE_PATH     = "${REMOTE_BASE}/releases/${RELEASE_NAME}"
        KEEP_RELEASES    = 5
        PHP_FPM_SERVICE  = 'php8.1-fpm'
    }

    options {
        timestamps()
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '20'))
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Composer Install') {
            steps {
                sh 'composer install --no-dev --optimize-autoloader --prefer-dist'
            }
        }

        stage('Chuan bi env.php gia de compile') {
            steps {
                sh 'cp app/etc/env.php.jenkins app/etc/env.php'
            }
        }

        stage('DI Compile') {
            steps {
                sh 'php bin/magento setup:di:compile'
            }
        }

        stage('Static Content Deploy') {
            steps {
                sh 'php bin/magento setup:static-content:deploy -f vi_VN en_US'
            }
        }

        stage('Dong goi artifact') {
            steps {
                sh '''
                    rm -f app/etc/env.php
                    tar --exclude="./pub/media" \
                        --exclude="./var/log" \
                        --exclude="./var/cache" \
                        --exclude="./var/page_cache" \
                        --exclude="./var/session" \
                        --exclude="./.git" \
                        -czf release.tar.gz .
                '''
            }
        }

        stage('Tao thu muc release tren VPS') {
            steps {
                sshagent(credentials: ['jenkins-deploy-key']) {
                    sh "ssh -o StrictHostKeyChecking=no ${DEPLOY_HOST} 'mkdir -p ${RELEASE_PATH}'"
                }
            }
        }

        stage('Upload va giai nen') {
            steps {
                sshagent(credentials: ['jenkins-deploy-key']) {
```

```

        sh """
            scp -o StrictHostKeyChecking=no release.tar.gz ${DEPLOY_HOST}:${RELEASE_PATH}/
            ssh ${DEPLOY_HOST} 'tar -xzf ${RELEASE_PATH}/release.tar.gz -C ${RELEASE_PATH} && rm ${RELEASE_PATH}/release.tar.gz'
        """
    }
}

stage('Link Shared Resources') {
    steps {
        sshagent(credentials: ['jenkins-deploy-key']) {
            sh """
                ssh ${DEPLOY_HOST} '
                    ln -sfn ${REMOTE_BASE}/shared/app/etc/env.php ${RELEASE_PATH}/app/etc/env.php
                    rm -rf ${RELEASE_PATH}/pub/media
                    ln -sfn ${REMOTE_BASE}/shared/pub/media ${RELEASE_PATH}/pub/media
                    rm -rf ${RELEASE_PATH}/var/log
                    ln -sfn ${REMOTE_BASE}/shared/var/log ${RELEASE_PATH}/var/log
                '
            """
        }
    }
}

stage('Enable Maintenance Mode') {
    steps {
        sshagent(credentials: ['jenkins-deploy-key']) {
            sh "ssh ${DEPLOY_HOST} 'cd ${RELEASE_PATH} && php bin/magento maintenance:enable'"
        }
    }
}

stage('DB Upgrade & Reindex') {
    steps {
        sshagent(credentials: ['jenkins-deploy-key']) {
            sh """
                ssh ${DEPLOY_HOST} '
                    cd ${RELEASE_PATH}
                    php bin/magento setup:upgrade --keep-generated
                    php bin/magento indexer:reindex
                    php bin/magento cache:flush
                '
            """
        }
    }
}

stage('Switch Symlink (Go Live)') {
    steps {
        sshagent(credentials: ['jenkins-deploy-key']) {
            sh "ssh ${DEPLOY_HOST} 'ln -sfn ${RELEASE_PATH} ${REMOTE_BASE}/current'"
        }
    }
}

stage('Disable Maintenance & Reload Services') {
    steps {
        sshagent(credentials: ['jenkins-deploy-key']) {
            sh """
                ssh ${DEPLOY_HOST} '
                    cd ${REMOTE_BASE}/current
                    php bin/magento maintenance:disable
                    sudo service ${PHP_FPM_SERVICE} reload
                    sudo systemctl reload nginx
                '
            """
        }
    }
}

stage('Fix Permissions') {
    steps {
        sshagent(credentials: ['jenkins-deploy-key']) {
            sh "ssh ${DEPLOY_HOST} 'sudo chown -R www-data:www-data ${RELEASE_PATH}'"
        }
    }
}

stage('Cleanup Old Releases') {
    steps {
        sshagent(credentials: ['jenkins-deploy-key']) {
            sh """
                ssh ${DEPLOY_HOST} '
                    cd ${REMOTE_BASE}/releases
                    ls -lt | tail -n +${KEEP_RELEASES+1} | xargs -r rm -rf
                '
            """
        }
    }
}

post {
    failure {
        echo 'Deploy that bai - kiểm tra Console Output, cần nhac rollback bang job riêng.'
    }
}

```

```

    }
    success {
        echo "Deploy thành công: release ${RELEASE_NAME}"
    }
    always {
        sh 'rm -f release.tar.gz'
    }
}
}

```

Commit và push:

```

git add Jenkinsfile
git commit -m "Add Jenkinsfile for Magento CI/CD pipeline"
git push origin main

```

Xác nhận Jenkins job trở đúng: **Configure → Pipeline → Script Path = Jenkinsfile**.

9. Giải thích các stage quan trọng

9.1. Link Shared Resources

```

ln -sfn ${REMOTE_BASE}/shared/app/etc/env.php ${RELEASE_PATH}/app/etc/env.php
rm -rf ${RELEASE_PATH}/pub/media
ln -sfn ${REMOTE_BASE}/shared/pub/media ${RELEASE_PATH}/pub/media
rm -rf ${RELEASE_PATH}/var/log
ln -sfn ${REMOTE_BASE}/shared/var/log ${RELEASE_PATH}/var/log

```

Mỗi release mới không chứa **env.php** (bí mật production), ảnh **pub/media** (dữ liệu khách hàng upload), hay **var/log** (lịch sử log). Symlink giúp mọi release **dùng chung một nguồn dữ liệu thật duy nhất** nằm ở **shared/**, thay vì copy lại mỗi lần deploy.

Ý nghĩa cờ **-sfn**: **-s**: tạo symbolic link (không copy) - **-f**: ghi đè nếu đã tồn tại - **-n**: thay thế chính symlink đích, không đi theo symlink cũ

rm -rf trước khi **ln -s** cho **pub/media** và **var/log** là bắt buộc vì sau khi giải nén **.tar.gz**, các thư mục này vẫn tồn tại (rỗng hoặc có file mặc định) — cần xóa trước để **ln -s** không báo lỗi.

Không symlink **var/cache**, **var/session**, **generated/** vì đây là dữ liệu sinh ra từ chính code mỗi release, dùng chung sẽ gây xung đột giữa các version.

9.2. Vì sao vẫn cần Maintenance Mode

1. Upload code → release mới (chưa live, an toàn)
2. Link Shared Resources (chưa live)
3. Enable Maintenance Mode <- bắt đầu downtime
4. DB Upgrade & Reindex <- đang downtime
5. Switch Symlink (Go Live) <- đổi symlink, tức thời
6. Disable Maintenance <- kết thúc downtime

Nếu switch symlink **trước** khi chạy **setup:upgrade**, code mới sẽ đọc/ghi theo schema DB cũ → lỗi. Nếu chạy **setup:upgrade** **sau** khi switch, code cũ sẽ không tương thích schema mới trong lúc chờ. Do đó cần một khoảng bảo trì ngắn để đảm bảo code và database luôn đồng bộ — đây là đánh đổi hợp lý cho một VPS đơn, khác với zero-downtime tuyệt đối (cần hạ tầng nhiều server).

9.3. Cleanup Old Releases

```

cd ${REMOTE_BASE}/releases
ls -lt | tail -n +${(( ${KEEP_RELEASES} + 1 ))} | xargs -r rm -rf

```

- **ls -lt**: liệt kê thư mục release, mới nhất lên đầu
- **tail -n +6** (khi **KEEP_RELEASES=5**): in ra từ dòng thứ 6 trở đi — tức các release cũ hơn 5 bản gần nhất
- **xargs -r rm -rf**: xóa các thư mục đó; cờ **-r** đảm bảo không chạy **rm** nếu danh sách rỗng (tránh lỗi khi mới deploy lần đầu)

Chạy **sau** khi switch symlink để đảm bảo không bao giờ xóa nhầm release đang live.

Dấu **\\$** (có backslash) trong Jenkinsfile là bắt buộc — báo cho Groovy đừng nội suy **\${(...)}**, để nguyên cú pháp cho Bash xử lý khi lệnh chạy trên VPS.

10. Job Rollback

New Item → Pipeline → tên **rollback-xgear**:

```
pipeline {
    agent any
    parameters {
        string(name: 'RELEASE_TO_ROLLBACK', defaultValue: '', description: 'De trong = quay ve release lien truoc')
    }
    environment {
        DEPLOY_HOST = 'deploy@<VPS_APP_IP>'
        REMOTE_BASE = '/var/www/xgear-deploy'
    }
    stages {
        stage('Rollback') {
            steps {
                sshagent(credentials: ['jenkins-deploy-key']) {
                    sh '''
                        ssh ${DEPLOY_HOST} '
                        cd "${REMOTE_BASE}"/releases
                        TARGET=""${RELEASE_TO_ROLLBACK}"
                        if [ -z "$TARGET" ]; then
                            TARGET=$(ls -lt | sed -n 2p)
                        fi
                        echo "Rolling back to: $TARGET"
                        ln -sf "${REMOTE_BASE}"/releases/$TARGET "${REMOTE_BASE}"/current
                        sudo service php8.1-fpm reload
                    '''
                }
            }
        }
    }
}
```

Vì `current` chỉ là symlink, rollback diễn ra trong vài giây. Nên chạy thử job này ít nhất 1 lần ngay sau khi pipeline chính hoạt động ổn định, để chắc chắn nó hoạt động đúng trước khi cần dùng thật trong sự cố.

11. Cloudflare — purge cache sau deploy (tùy chọn)

Lấy `Zone ID` (Cloudflare Dashboard → chọn domain → Overview) và tạo API Token (My Profile → API Tokens → quyền Cache Purge).

Thêm Secret text credential trong Jenkins (`cf-api-token`), rồi thêm stage vào cuối Jenkinsfile:

```
stage('Purge Cloudflare Cache') {
    steps {
        withCredentials([string(credentialsId: 'cf-api-token', variable: 'CF_TOKEN')]) {
            sh """
                curl -s -X POST "https://api.cloudflare.com/client/v4/zones/<ZONE_ID>/purge_cache" \
                -H "Authorization: Bearer ${CF_TOKEN}" \
                -H "Content-Type: application/json" \
                --data '{"purge_everything":true}'
            """
        }
    }
}
```

12. Bảng lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
<code>composer: command not found</code>	Chưa cài Composer trên máy local	<code>curl -sS https://getcomposer.org/installer</code> <code> php && sudo mv composer.phar</code> <code>/usr/local/bin/composer</code>
<code>Permission denied (publickey)</code> khi SSH	Credential ID sai hoặc key chưa copy đúng VPS	Test tay <code>ssh -i ~/jenkins_deploy_key</code> <code>deploy@<VPS_APP_IP></code>
<code>sudo: no tty present</code>	Lệnh sudo không chạy được qua non-interactive SSH	Đảm bảo <code>/etc/sudoers.d/deploy</code> có <code>NOPASSWD:</code>
<code>setup:di:compile</code> lỗi kết nối DB	<code>env.php.jenkins</code> cấu hình sai hoặc DB dummy chưa tồn tại	Cài MariaDB local + DB dummy thật (mục 7)
<code>Class not found</code> lúc compile	Cache cũ còn sót	Xóa <code>generated/</code> , <code>var/cache/*</code> trước khi compile
<code>scp: Permission denied</code> khi upload	User <code>deploy</code> không có quyền ghi <code>releases/</code>	<code>sudo chown -R deploy:www-data</code> <code>/var/www/xgear-deploy</code>
Site load nhưng lỗi 500 sau switch symlink	Thiếu quyền đọc file hoặc thiếu <code>var/generation</code>	<code>tail -f /var/www/xgear-</code> <code>deploy/current/var/log/exception.log</code>
<code>memory_limit</code> khi composer install	PHP CLI giới hạn RAM quá thấp	Sửa <code>/etc/php/8.1/cli/php.ini</code> : <code>memory_limit</code> <code>= -1</code>

13. Checklist trước khi Build Now lần đầu

- ☐ Đã backup code + DB trên VPS
- ☐ VPS: cấu trúc `releases/shared/current` đã tạo, `env.php` + `pub/media` đã copy sang `shared/`
- ☐ Nginx root/ `$MAGE_ROOT` VPS đã trỏ `current` , `nginx -t` OK, site vẫn load được
- ☐ SSH key `jenkins_deploy_key` test tay thành công
- ☐ Credential `jenkins-deploy-key` đã thêm đúng ID vào Jenkins
- ☐ File `app/etc/env.php.jenkins` đã commit, `env.php` thật đã vào `.gitignore` (kiểm tra bằng `git check-ignore -v`)
- ☐ `Jenkinsfile` đã push lên đúng branch đang build
- ☐ Jenkins job: Poll SCM đã bật với schedule hợp lý (`H/5 * * * *`)
- ☐ VPS sudoers đã cấp `NOPASSWD` cho reload php-fpm, reload nginx, chown

14. Vận hành sau khi build thành công

```
# Xác nhận symlink current
ssh deploy@<VPS_APP_IP> "ls -la /var/www/xgear-deploy/current"

# Kiểm tra site
curl -I https://<DOMAIN>

# Theo dõi log lỗi nếu cần
ssh deploy@<VPS_APP_IP> "tail -50 /var/www/xgear-deploy/current/var/log/exception.log"
```

Kiểm tra trực quan: trang chủ, ảnh sản phẩm (test symlink `pub/media`), trang `/admin` .

Tài liệu này là phần 1 trong 2 phần — phần 2 sẽ hướng dẫn chuyển Jenkins sang chạy trực tiếp trên VPS, kết nối GitHub Webhook thật (build tức thời khi push, không cần chờ Poll SCM).